

Machine Learning

Instructor: Hafiz Abdul Rehman

Lecture 11: Random Forest

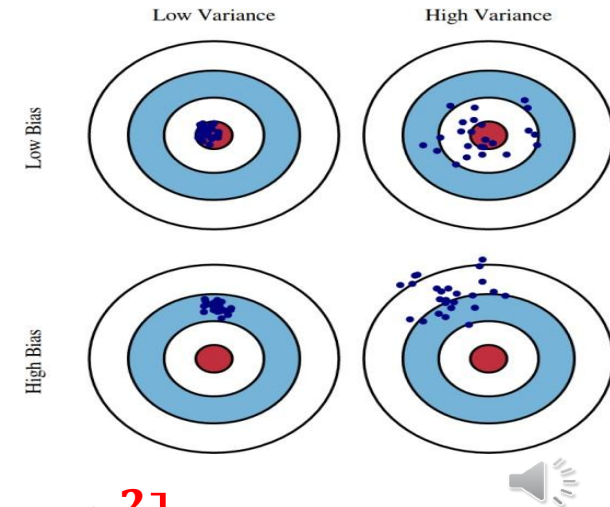
Setting Our Goal

$$E_{x,y,D}[(h_D(x) - y)^2] = \mathbf{E_{x,D}}[(\mathbf{h_D(x)} - \mathbf{\bar{h}(x)})^2] + E_{x,y}[(\bar{y}(x) - y)^2] + \mathbf{E_x}[(\mathbf{\bar{h}(x)} - \mathbf{\bar{y}(x)})^2]$$

- Where is the expected classifier (given ML algorithm A):

$$\bar{h} = E_{D \sim P^n} [h_D] = \int_D h_D \Pr(D) \partial D$$

- where $\Pr(D)$ is the probability of drawing D from P^n
- Here, $\bar{h}$ is a weighted average over functions h_D



- Goal:** Reduce the variance term: $\mathbf{E_{x,D}}[(\mathbf{h_D(x)} - \mathbf{\bar{h}(x)})^2]$
 - By making $h_D \rightarrow \bar{h}$

Using the weak law of large numbers

- For i.i.d. random variables x_i with mean $\bar{x}_i$, the weak law roughly states:

$$\frac{1}{m} \sum_{i=1}^m x_i \rightarrow \bar{x} \text{ as } m \rightarrow \infty$$

- **In case of classifiers:**

- Assume we have m training sets $D_1, D_2, \dots, D_m$ drawn from P
- Train a classifier on each one and average out the results

$$\hat{h} = \frac{1}{m} \sum_{i=1}^m h_{D_i} \rightarrow \bar{h} \text{ as } m \rightarrow \infty$$

- Such an average of multiple classifiers is referred to as an ensemble of classifiers.
- **Success!** $\square$ If $\hat{h} \rightarrow \bar{h}$, the variance component of the error must also vanish

$$E_{x, D} \left[\left(h_D(x) - \bar{h}(x) \right)^2 \right]$$

- **Ouch!** $\square$ We don't have m data sets. We only have our lonely D .

Bagging (Bootstrap Aggregating)

- **Simulate** drawing from P by drawing **uniformly with replacement** from the set D
 - To get $D_1, D_2, \dots, D_m$ – each one with the same size $|D|$
 - Use these datasets to train m classifiers, $h_1, h_2, \dots, h_m$
 - completely all the way to minimum error on the training data – i.e. overfitting
 - Finally average out the decisions of these m classifiers (in case of regression) or take the mode (majority vote, in case of classification)
- Okay, so now, can we claim that $\hat{h} \rightarrow \bar{h}$ using the weak law of large numbers?
 - Nope!
 - We cannot use W.L.L.N here
 - W.L.L.N, only works for i.i.d. samples and here we violate the independence assumption - knowing a few of D 's tells us about D_{i+1} as all of these come from the same dataset D containing the same finite set of points
- Can we even show that these points have been drawn from the same distribution P ?
 - Yes!
- However, **in practice** bagging still reduces variance **very effectively**.
 - If each of $h_1, h_2, \dots, h_n$ overfitted to some unique aspects of $D_1, D_2, \dots, D_n$, the average classifier will average out these over-reactions and preserve the commonalities across them that supposedly represent D , and in turn, P

Summary of Bagging

1. Sample m data sets $D_1, \dots, D_m$ from D with replacement.
 2. For each D_j train a classifier $h_j()$
 3. The final classifier is $h(x) = \sum_{j=1}^m h_j(x)$
- In practice, larger m results in a better ensemble
 - However, at some point we will obtain diminishing returns
 - Setting m unnecessarily high will only slow down the classifier but will not increase the error of the classifier.
 - Bagging is a general technique for reducing variance and not just specific to decision trees

Advantages of Bagging

- Easy to implement
- Reduces variance, so it is great for high variance classifiers.
- As the prediction is an average (or mode) of many classifiers, we obtain a mean and variance.
 - The variance can be interpreted as the uncertainty of the prediction. Especially in regression tasks, such uncertainties are otherwise hard to obtain.
 - Imagine the prediction of a house price is \$300,000. If a buyer wants to decide how much to offer, it would be very valuable to know the standard deviation of the prediction.

Advantages of Bagging

- Bagging provides an unbiased estimate of the test error – the **out-of-bag error**.
 - Take each training point (x_i, y_i) that was not picked by the data sets D_k . If we average the classifiers h_k of all such data sets, we obtain a classifier (with a slightly smaller m) that was not trained on (x_i, y_i)
 - It is therefore equivalent to a test sample.
 - Compute the error of all these classifiers to obtain an estimate of the true test error.
 - The beauty is that we can do this without reducing the training set. We just run bagging as it is intended and obtain this so called out-of-bag error for free.

- For each training point $(x_i, y_i) \in D$ let $S_i = \{k | (x_i, y_i) \notin D_k\}$ – in other words S_i is a set of all the training sets D_k , which do not contain (x_i, y_i) .
- Let the averaged classifier over all these data sets be

$$\bar{h}_i(x) = \frac{1}{|S_i|} \sum_{k \in S_i} h_k(x)$$

- The-of-bag error becomes simply the average error/loss that all these classifiers yield

$$\epsilon_{OUB} = \sum_{(x_i, y_i) \in D} l(\bar{h}_i(x) - y)$$

- This is an estimate of the test error, because for each training point, we used the subset of classifiers that never saw that training point during training.
- If m is sufficiently large, the fact that we take out some classifiers has no significant effect, and the estimate is reliable.

Random Forest

Advantages of Bagging

- One of the most famous and useful bagged algorithms is the **Random Forest**
 - A Random Forest is essentially just bagged decision trees, with a slightly modified splitting criteria.
- **Algorithm**
 1. Sample m data sets $D_1, \dots, D_m$ from D with replacement
 2. For each D_j train a full decision tree $h_j()$ ($\text{max-depth} = \infty$) with one small modification:
 - Before each split randomly subsample $k \leq d$ features (without replacement) and only consider these for your split
 - This further increases the variance of the trees – it makes the tree more different. Now each makes slightly different mistakes, and these get averaged out later
- The final classifier is $h(x) = \sum_{j=1}^m h_j(x)$

Benefits

- The Random Forest is one of the best, most popular and easiest to use out-of-the-box classifier
- The RF has only **two hyper-parameters**, m and k
 - It is extremely insensitive to both
 - A good choice for k is $k = \sqrt{d}$ (where d denotes the number of features).
 - Set m as large as you can afford
- All benefits of trees get passed down
 - Decision trees do not require a lot of preprocessing. **No need to normalize/scale features for mixture of feature types and ranges**
 - For example, the features can be of different scale, magnitude, or slope. This can be highly advantageous in scenarios with heterogeneous data, for example the medical settings where features could be things like blood pressure, age, gender, ..., each of which is recorded in completely different units.
- **Feature selection:** When we split on a feature, how much does the entropy go down. Take average across trees in the forest and sort.
- **Resilient to curse-of-dimensionality**
- Out-of-bag error as a great estimator of test error – no need for training/validation splits. Train on the entire dataset!

Variants

- Split each training set into two partitions $D_l = D^A \cup D^B$, where $D^A \cap D^B = \emptyset$.
 - Build the tree on D^A and estimate the leaf labels on D^B . You must stop splitting.
 - if a leaf has only a single point in D^B in it. This has the advantage that each tree and also the RF classifier become consistent.
- Do not grow each tree to its full depth, instead prune based on the left-out samples (OOB error).
 - This can further improve bias/variance trade-off

Thank You